

IK141 Struktur Data

Singly Linked List



Disusun Oleh:

Muhamad Akbar Imron	Ryan Yanuar Pradana	M. Raditya Santosa
Aura Putri Kireina	Reybano Ivander Lucky P.	Salsabila Ramadhani H.

2990 – Erna Piantari, M.T

PROGRAM STUDI PENDIDIKAN ILMU KOMPUTER

FAKULTAS PENDIDIKAN MATEMATIKA DAN ILMU PENGETAHUAN ALAM

UNIVERSITAS PENDIDIKAN INDONESIA

1. Identitas

Nama Mata Kuliah – Kode	IK141 – Struktur Data
Materi	Singly Linked List
Sub Materi	- Create - Insert Awal & Akhir - Insert Tengah Sebelum & Sesudah - Update - Delete Awal & Akhir - Delete Tengah - Print
Beban (Waktu Pelaksanaan)	150 menit
Semester	2
Pengampu MK	Erna Piantari, M.T

2. Capaian Praktikum

Capaian Praktikum
- Menguasai konsep teoritis ilmu komputer dalam kaitannya pada pemecahan masalah melalui perancangan

3. Perangkat yang Dibutuhkan

Perangkat lunak
- Compiler bahasa C - Pada windows : Visual Studio

Ringkasan Materi
1. Konsep Update pada Single Linked List Pada <i>Single Linked List</i> , operasi update berarti memperbarui sebuah data pada <i>node</i> dari <i>list</i> . Misalnya, kita punya linked list:

2990 – Erna Piantari, M.T

PROGRAM STUDI PENDIDIKAN ILMU KOMPUTER

FAKULTAS PENDIDIKAN MATEMATIKA DAN ILMU PENGETAHUAN ALAM

UNIVERSITAS PENDIDIKAN INDONESIA



MODUL PRAKTIKUM

IK141 – Struktur Data

1) Cek apakah list kosong

Case:

Maka **tidak ada** data pada node yang diperbarui..

2) Cek apakah list hanya ada 1 node

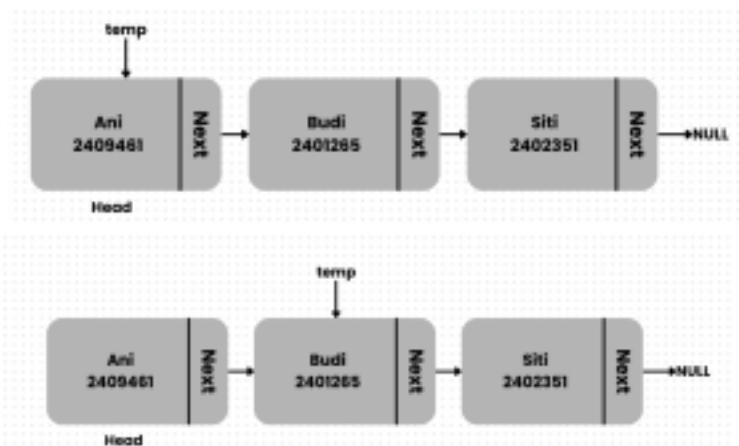
Karena hanya ada 1 node, maka:

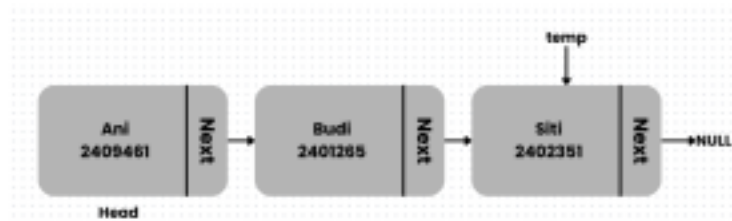
- Cek apakah data pada node tersebut yang dicari,
- **Jika iya**, maka perbarui data pada node tersebut,
- **Jika tidak**, maka tidak ada data pada node yang diperbarui.

3) Update data

Kita perlu mencari terlebih dahulu data pada *node* mana yang ingin diperbarui. Misalnya, kita ingin **memperbarui** nama Siti menjadi **Siti Ayu** dengan **kunci pencariannya** adalah nim Siti, yaitu 2402351.

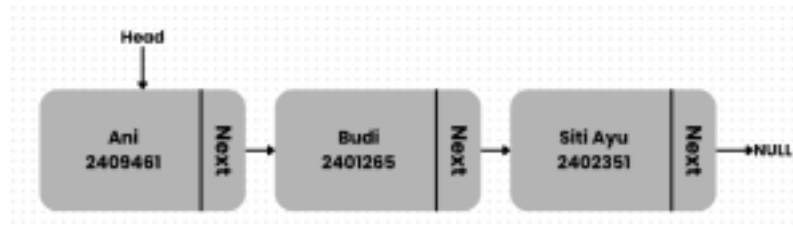
Maka perulangannya selama $\text{temp} \rightarrow \text{next} \neq \text{NULL}$ **dan** kondisi jika $\text{temp} \rightarrow \text{nim} \neq \text{nimCari}$, dengan contoh kasus *linked list* di atas, perulangan **temp** akan berhenti di Siti.





Jika **kunci pencarian** sudah ditemukan, maka perbarui data menggunakan *string copy* karena yang diperbarui adalah namanya.

4) Linked list setelah update



Dengan penjelasan langkah-langkah di atas, maka dapat dibuat prosedur seperti ini:

```

1 void update(char nimCari[], char newNama[]){
2     struct Node *temp = head;
3     while(temp != NULL){
4         if(strcmp(temp->nim, nimCari) == 0){
5             strcpy(temp->nama, newNama);
6             return;
7         }
8         temp = temp->next;
9     }
10    printf("Data dengan NIM %s tidak ditemukan.\n", nimCari);
11 }
  
```

2. Konsep Delete pada Single Linked List

Pada *Single Linked List*, operasi **delete** berarti menghapus sebuah *node* dari list. Karena *node* cuma punya pointer ke node berikutnya, kita harus hati-hati mengatur ulang pointer. Untuk menghapus sebuah *node*, digunakanlah sintaks **free()**.

Jenis Operasi **Delete**:

a) Delete di awal (head)

Delete di awal adalah proses menghapus *node* paling awal (*head*) dalam single linked list. Maka dari itu kita harus:

- Cek apakah linked list kosong,
- Menyimpan *node* awal (*head*) ke variabel sementara (*temp*),

- Memindahkan *head* ke node berikutnya,
 - Menghapus node awal dari memori.
- Misalnya kita punya linked list:

1) Cek apakah list kosong

Case:

Maka **tidak ada** node yang dihapus.

2) Cek apakah list hanya ada 1 node

Karena hanya ada 1 node, maka :

- Hapus node tersebut
- *Set head* == NULL

3) Cek apakah list hanya ada 1 node

Kita tidak langsung menghapus *head* walaupun ini DeleteAwal. Tetapi, pindahkan terlebih dahulu posisi *head* ke node selanjutnya agar posisi *head* tidak kosong. Karena posisi *head* **tidak boleh** kosong.

Maka, perlu variabel tambahan yaitu, **temp** yang menyimpan nilai *head* lalu **head = (head)->next**. Sehingga posisi *head* pindah ke *node* selanjutnya.

Maka, posisi



head yang terbaru akan pindah ke *node* selanjutnya.

4) Hapus node pertama

Hapus node pertama (*temp*) menggunakan sintaks:



5) Linked list setelah deleteAwal



Dari penjelasan langkah-langkah diatas, maka dapat dibuat prosedur seperti ini:

2990 – Erna Piantari, M.T

PROGRAM STUDI PENDIDIKAN ILMU KOMPUTER

FAKULTAS PENDIDIKAN MATEMATIKA DAN ILMU PENGETAHUAN ALAM

UNIVERSITAS PENDIDIKAN INDONESIA

MODUL PRAKTIKUM

IK141 – Struktur Data



b) Delete di akhir (*tail*)

Delete akhir adalah proses menghapus *node* terakhir dalam *single linked list*. Karena tidak ada pointer ke belakang, kita harus:

- Mencari *node* sebelum terakhir,
- Memutus sambungan ke *node* terakhir,
- Lalu menghapus *node* terakhir dari memori.

Misalnya kita punya linked list:

Target: hapus *node* terakhir → Siti | 2402351

1) Cek Apakah List Kosong

Case:

Maka **tidak ada** *node* yang dihapus.

2) Cek Apakah List Hanya 1 Node

Karena hanya satu *node*:

- Hapus *node* tersebut
- Set head = NULL

3) Cari Node Sebelum Terakhir

Kita tidak berhenti di *node* terakhir, tapi di *node* sebelum terakhir. Maka perulangannya selama $\text{temp} \rightarrow \text{next} \neq \text{NULL}$, dengan contoh kasus *linked list* di atas, perulangan **temp** akan berhenti di Budi.



4) Putuskan Sambungan ke Node Terakhir

Awalnya, $\text{temp} \rightarrow \text{next}$ adalah Siti dan $\text{temp} \rightarrow \text{next} \rightarrow \text{next}$ adalah NULL.

Kita hapus data $\text{temp} \rightarrow \text{next}$ dan jadikan $\text{temp} \rightarrow \text{next} = \text{NULL}$ menggunakan sintaks:



5) Linked List Setelah Delete

Hasil akhir:



Dari penjelasan langkah-langkah diatas, maka dapat dibuat prosedur seperti ini:



c) Delete Tengah (berdasarkan nilai)

Delete tengah dilakukan jika ingin menghapus *node* yang diinginkan. Untuk itu kita harus:

- Mencari *node* yang diinginkan,
- Menyimpan *node* sebelumnya,
- Menghubungkan *node* sebelumnya ke *node* setelah *node* yang dihapus,
- Menghapus *node* yang dihapus dari memori,

Misalnya kita punya linked list:

Target: hapus *node* → Budi | 2401265

1) Cek apakah List Kosong

Case:

Maka **tidak ada** *node* yang dihapus.

2) Cek apakah NIM yang dicari adalah bukan Head

Jika NIM yang dicari adalah **head** yang **ditandakan** oleh `prev = NULL`, maka program bisa **mengarahkan** untuk `deleteAwal`.

3) Cek apakah NIM yang dicari adalah bukan Tail

Jika NIM yang dicari adalah **tail** yang **ditandakan** oleh `temp->next = NULL`, maka program bisa **mengarahkan** untuk `deleteAkhir`.

4) Inisialisasi Pointer

Kita pakai **dua** pointer:

- `temp` → menunjuk *node* yang sedang dicek
- `prev` → menyimpan *node* sebelumnya



5) Cari node yang akan dihapus

Kita telusuri list sampai NIM ditemukan yaitu `temp->nim = nim`.



Sekarang `temp` adalah *node* Budi dan `prev` adalah *node* Ani.

6) Putuskan dan Sambungkan Pointer

Awalnya `temp->next` adalah *node* Siti adalah `prev->next` adalah *node* Budi.

Karena kita ingin menghapus Budi, maka sambungkan *node* Ani ke *node* Siti menggunakan sintaks:



7) Hapus Node Dari Memori

Karena *node* sudah dilepas dari list, maka hapus *node* Budi dengan menggunakan sintaks:



8) Linked List setelah Delete

Hasil akhir:



Dari penjelasan langkah-langkah diatas, dapat dibuat menjadi prosedur seperti ini:

2990 – Erna Piantari, M.T

PROGRAM STUDI PENDIDIKAN ILMU KOMPUTER

FAKULTAS PENDIDIKAN MATEMATIKA DAN ILMU PENGETAHUAN ALAM

UNIVERSITAS PENDIDIKAN INDONESIA

MODUL PRAKTIKUM

IK141 – Struktur Data



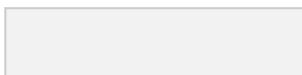
2990 – Erna Piantari, M.T

PROGRAM STUDI PENDIDIKAN ILMU KOMPUTER

FAKULTAS PENDIDIKAN MATEMATIKA DAN ILMU PENGETAHUAN ALAM

UNIVERSITAS PENDIDIKAN INDONESIA

MODUL PRAKTIKUM



3. Implementasi Main

Dari keseluruhan penjelasan di atas, maka pemanggilan prosedur yang diperlukan dapat seperti ini:



2990 – Erna Piantari, M.T

PROGRAM STUDI PENDIDIKAN ILMU KOMPUTER

FAKULTAS PENDIDIKAN MATEMATIKA DAN ILMU PENGETAHUAN ALAM

UNIVERSITAS PENDIDIKAN INDONESIA